Recall-Bounded Durability for Graph-Based Approximate Nearest Neighbor Indexes

Habib Rahman
Toronto Metropolitan University
Toronto, Canada
Meta
Menlo Park, California, USA
habib.rahman@torontomu.ca

ABSTRACT

Approximate nearest-neighbor (ANN) work usually reports recall and queries per second, not whether an acknowledged insert survives failure. We implement an opt-in committer whose strict policy caps post-crash top-k membership loss by ε for every eligible query; the shipped default remains durable-before-return. Versioned WAL fences, cooperating-writer exclusion, an exact volatile tail isolated from HNSW topology, and stable-only fallback make the bound auditable after production recovery.

The central result is a tradeoff, not an unconditional speedup. At $k = 10$, strict commit reaches median 251/252 writes/s on random/hot workloads versus 326 for stable group commit. Exchangeable-mean admission reaches 493/511 writes/s but drops the per-query guarantee and loses 0.225 recall on hot inserts. In a paired $\varepsilon = .2$ sweep, strict remains slower at $k = 10$, but at $k = 100$ its 20-record cap binds and yields median strict/stable ratios of 1.61/1.64; strict is faster in all 30 within-fixed-graph timing repetitions for each workload. A hard guarantee therefore costs throughput at small k and is faster here only when its allowed weak batch amortizes fencing.

Across 15 process-crash frontiers, 661 WAL byte prefixes, and 2,880 queries from 240 reopened images, stable records never disappear, admission never overshoots, and strict observations satisfy $\Delta^+ \le L \le M \le \varepsilon = .2$, where Δ^+ is positive recall drop, L actual lost top-k membership, and M volatile-tail exposure. On one CentOS 9/ext4 host, a separate dm-log-writes run also passes all 102 recorded block-log cuts: every stable acknowledgment survives and any permitted absence is confined to the weak window. A separate two-host study quantifies synchronization, batching, and recovery costs. The evaluation remains single-node and insert-only; this device-level result covers one host, filesystem, and synthetic workload.

PVLDB Reference Format:
Habib Rahman. Recall-Bounded Durability for Graph-Based Approximate Nearest Neighbor Indexes. PVLDB, 19(XX): XXX–XXX, 2026.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Graph-based ANN indexes—Hierarchical Navigable Small World (HNSW) graphs [11] and their descendants—dominate in-memory vector search, and vector databases built on them now back retrieval-augmented generation and semantic search at scale. The community evaluates them on *recall@k* (the fraction of a query's true k nearest neighbors the index returns) and *throughput* (queries/s, QPS), summarized by the recall-vs-QPS Pareto frontier popularized by ANN-Benchmarks [1] and the Big ANN challenges [13]. What these evaluations almost never measure is *durability*: does an acknowledged write survive a crash?

Vector databases now ingest continuously, yet mainstream ANN libraries omit durable storage (Faiss offers manual serialization [4]; hnswlib has no crash contract [10]), and common benchmarks omit crash recovery and fsync [1, 13, 24]. Production systems such as Milvus [19] use WALs but publish no isolated durability analysis. Research systems either inherit a host DB's durability [18] or, like LSM-VEC [23], report no synchronization or recovery cost.

Our main finding is not that weak acknowledgment is universally faster. At the default operating point, enforcing a hard per-query loss bound is slower than stable group commit. The higher-throughput configuration replaces that bound with an exchangeability assumption and fails on query-hot inserts. We therefore report the guarantee's cost first, then ask separately whether larger result sets let strict fencing amortize enough to reverse the ordering. The paired sweep finds a clear reversal by $k_{\min} = 50$, strongest at 100, but not at the default $k = 10$.

We use established storage techniques in a standalone graph-ANN index; group commit, snapshots, WAL fencing, and the exact-tail composition are not new algorithms. Our contribution is an auditable post-recovery loss-cap contract and an evidence methodology that checks it across every latest-query path, with stable-only fallback where required (§6). Legacy measurements span Apple and AWS Linux; committer timings use one Apple host, while the device-level replay uses one AWS CentOS 9/ext4 host.

Contributions.

- **C1 (§3).** An implementation and audit of established crash-consistency techniques on an ANN path, with tmpfs detection, a sync-floor baseline, and explicit fsync-mode labeling.
- **C2 (§4).** An illustrative pre-committer mutable-HNSW durability-tax measurement versus durability strength, with an AWS Linux cross-check.
- **C3 (§5).** A pre-committer mutable-HNSW group-commit *batch-size sweep* showing throughput versus batch size and fsync mode.
- **C4 (§7).** A pre-committer baseline comparing snapshot load with graph-rebuilding WAL replay on block

storage, complementing P-HNSW's persistent-memory analysis [9].

- **C5 (§6).** An auditable weak-ACK contract with a strict distribution-free top-k loss cap, enforced across all latest-query paths and checked after production recovery and device-level block replay. We measure the price of that guarantee against stable and model-conditional alternatives.

2 BACKGROUND AND SYSTEM

Our testbed is a from-scratch C++20 vector database with a segmented store. Its query representation has three layers: immutable sealed HNSW snapshots; a durable raw delta R; and, only when opt-in weak acknowledgment is enabled, a volatile raw tail U. Latest reads search all three layers, stable reads omit U, and the results are merged by the deterministic key $(\mathrm{distance}, \mathrm{key})$. Both raw layers use exact distance scans. In particular, inserting into U and later hardening it into R never mutates an HNSW. The existing insert API remains stable-by-default and returns only after synchronization. When recall commit is enabled, every latest-search API—scalar, metadata, and batch—serves $k < k_{\min}$ from the stable snapshot and reports the effective visibility on the structured response. Thus every query that can observe U lies inside the admission contract.

Durability model. Version-2 WAL mutation frames contain explicit lengths, an LSN, operation, payload, and a header-plus-payload CRC. A fence covers a generation by recording its LSN range, mutation count, rolling hash, and its own CRC. Before any writable recovery, manifest update, high-water reservation, or WAL append, the process takes an exclusive OS lock on the storage root and holds it for the database lifetime. A second writer fails cleanly; read-only verification does not take the exclusive lock. Within the owning process, the sole WAL appender maintains $\mathrm{appended} \geq \mathrm{visible} \geq \mathrm{durable}$; frame append precedes in-memory publication internally, while current public status snapshots report $\mathrm{appended} = \mathrm{visible} \geq \mathrm{durable}$. It reserves LSN ranges in a CRC-protected high-water file before assignment, so discarded weak-tail LSNs are never reused. In strong macOS mode, high-water replacement full-syncs the temporary file, renames it, and then full-syncs the parent directory before the range becomes assignable; rollover fault injection verifies this ordering and fails the write before any receipt if the directory barrier fails. Recovery bounds lengths before allocation, validates mutations and fences, and accepts only complete fenced generations. Writable recovery truncates and synchronizes any suffix; the read-only verifier follows the same parse without modifying the image. Legacy version-1 WALs are treated as stable and upgraded before weak mode can be enabled. In strong macOS mode, a failed $\mathrm{F\_FULLFSYNC}$ is retried only for interruption and otherwise propagates; it never downgrades to plain fsync, and a failed fence cannot return a stable ACK.

Sealing is likewise crash-safe: it writes and synchronizes a snapshot plus $\mathrm{seal.ready}$, creates the replacement mutable generation, atomically publishes the manifest, and only then retires the old WAL. Failpoints cover each publication frontier. Recovery of a sealed segment loads its snapshot rather than rebuilding the graph from its WAL.

Index. Search uses HNSW with the Malkov–Yashunin diversifying neighbor-selection heuristic [11]. The search-time candidate-list width ef (and its build-time analogue $\mathrm{ef}_{\mathrm{construction}}$) trade recall for speed. Distance kernels are SIMD-vectorized (NEON on our host; an AVX+FMA path is implemented but untested here) with a devirtualized hot path. Recall@10 on SIFT1M rises with ef to 0.999 and is on the same recall–QPS frontier as hnswlib (Table 8), establishing a competitive index before durability is added.

3 MEASURING DURABILITY HONESTLY

Durability numbers are worthless without a statement of *what* was made durable. The pitfalls below are well known in the storage-systems literature— application-level crash-consistency is hard to get right [12], commodity stores mishandle power faults [22], and even PostgreSQL mishandled fsync error reporting [3]. Our contribution is not to discover them but to port this discipline to ANN evaluation, where it is absent. Two traps recur.

The tmpfs trap. Benchmarks frequently run under $/\mathrm{tmp}$, which on some Linux distributions is a RAM-backed tmpfs; synchronization there cannot demonstrate persistence across loss of power. Such an insert benchmark measures the memory-backed path only. Our harness statfs-checks the target and *refuses* to report write numbers on tmpfs.

The fsync*-mode trap.* Apple documents that $\mathrm{fsync}(2)$ does not necessarily flush a drive's volatile cache; $\mathrm{fcntl}(\mathrm{F\_FULLFSYNC})$ asks the device to complete that stronger operation. A number labeled "power-loss durable" under plain macOS fsync therefore overstates its contract. Our *sync floor*, a tight $\mathrm{write}()+\mathrm{sync}$ loop, separates device synchronization from index work. The $155\times$ mode gap in Table 1 is a macOS/APFS device microbenchmark, not a portable system speedup; the transferable result is the contract distinction.

Methodology and statistics. The legacy durability study reports the median and observed range of seven trials after warm-up; larger-N sweeps and the extended recovery experiment use three. Seven observations do not justify a parametric confidence interval, so differences smaller than overlapping ranges are treated as noise. The committer study is separate: it uses 30 executed tail/write timing repetitions per case and shuffles case order within each repetition. Its unit of analysis is an image: recovery queries are averaged within an image, then min/median/max are reported across images. We do not bootstrap correlated query rows. The images share one query set and two fixed base graphs (HNSW seeds 100 and 117); each case uses each seed 15 times. Six cases use a terminal unfenced-suffix frontier. Strict-random exits at its two-record cap before a fence; strict-hot exits after fence sync but before publication, then resumes verified work. These are timing repetitions, not independent workload or graph samples. SIFT1M recall is one fixed build per index, with single-run query timing only, so we do not call it deterministic across independently randomized graph builds.

Table 1: Sync-call floor in a tight write()+sync loop, median of 7 trials (min–max). Plain macOS/APFS fsync does not promise to force a volatile device cache, whereas F_FULLFSYNC requests that stronger operation; Linux fsync has the stronger storage-completion contract. The legacy loop did not record syscall return values, so these are timing measurements, not evidence of power-loss survival. The 155× ratio is the same-host macOS contract gap.

Host and requested mode	Calls/s, median [min,max]
macOS/APFS, plain fsync	49,199 [41,983, 51,782]
macOS/APFS, F_FULLFSYNC	317 [291, 324]
Linux/NVMe, fsync	527 [511, 529]

Table 2: Pre-committer mutable-HNSW durability tax (Apple M4 Pro, d=128, per-write durability, N=2,000 inserts/trial). Values are medians [min,max] over 7 trials; latency is milliseconds. Honest durability cuts throughput to ~22% (4.5×) and roughly triples p99.

Mode	Inserts/s	p50 (ms)	p99 (ms)
Plain fsync	442 [408,476]	1.73 [1.44,1.90]	14.4 [14.1,14.9]
F_FULLFSYNC	98 [98,106]	8.10 [7.97,8.23]	48.4 [35.3,49.4]

Cross-platform validation (Linux/NVMe). To confirm the phenomena are not macOS artifacts, we ran the legacy suite on an AWS Linux host (x86-64, ext4 on NVMe), where fsync requests the storage-completion contract used by the application. The stronger-contract per-write rate is 378 inserts/s (vs. the 527/s floor), group commit yields 4.9× (Table 3 shows 4.3× on macOS F_FULLFSYNC), and the recovery gap reaches 128× at N=6k with sealing cost (29–47 ms) again far below the WAL replay it saves (385–4843 ms). So the durability tax, the group-commit curve, and the snapshot-vs-replay recovery result all hold on the platform vector databases actually run on—the macOS numbers are illustrative, but these legacy tax and recovery trends are not confined to it. The new committer timing and process-crash measurements were run on the Apple host; a separate CentOS 9/ext4 block-replay experiment tests its durability contract below.

4 PRE-COMMITTER DURABILITY TAX

We define the *durability tax* as the throughput an index loses and the latency it adds to guarantee that acknowledged writes survive. We measure it by inserting one record at a time (each fsync'd before returning) into the archived pre-committer segmented store, where each mutable insert also mutated HNSW, and vary *only* the fsync mode. The SIFT-scale configuration uses d=128 and $\mathrm{ef}_{\mathrm{construction}}$=200. Because the two rows in Table 2 run the identical index work and differ only in whether the per-insert flush actually reaches the drive, the difference between them isolates the durability cost for that engine. Since commit 4017fa7, the current engine exact-scans a raw mutable delta and builds HNSW only at seal; current make bench-durability therefore does not reproduce these rows.

Table 3: Pre-committer mutable-HNSW group-commit batch-size sweep (Apple M4 Pro, d=128, N=2,000 inserts/trial, one fsync per batch), inserts/s as median of 7 trials (min–max), and speedup over batch 1. Under F_FULLFSYNC the speedup climbs a curve to 4.3×; under plain fsync batching barely helps because synchronization was already cheap in this measurement. Speedups are computed on unrounded medians.

	F_FULLFSYNC (honest)		Plain fsync	
Batch	Inserts/s	Speedup	Inserts/s	Speedup
1	102 (97–109)	1.0×	481 (415–491)	1.0×
10	339 (318–356)	3.3×	498 (453–533)	1.0×
50	402 (391–443)	4.0×	509 (462–560)	1.1×
200	419 (410–456)	4.1×	496 (443–563)	1.0×
1000	431 (411–461)	4.2×	514 (440–539)	1.1×
2000	433 (418–461)	4.3×	507 (447–543)	1.1×

Stronger durability cuts throughput to about a fifth and roughly triples p99. We do not subtract the single-sync floor: an insert performs multiple durable operations, so that floor is a lower bound rather than a clean latency component. Under plain fsync, graph insertion dominates; this both understates the stronger contract's cost and masks batching's benefit (§5).

Scale checks. In a three-trial size sweep, the plain-to-F_FULLFSYNC throughput ratio falls from 4.5× at N = 2k to 2.4× at 10k and 2.1× at 30k as graph work grows relative to fixed synchronization cost; group-commit benefit falls similarly. The small trial count and wide raw ranges make this a trend, not a fitted scaling law.

5 PRE-COMMITTER GROUP COMMIT FOR A GRAPH-ANN WAL

Group commit is a classic database technique [8]: batch several log records and issue a single fsync for the group, decoupling per-operation commit latency from flush latency. In the archived mutable-HNSW engine, we added a deferred-sync mode to the mutable segment: during a batch, WAL appends skip the per-record fsync, and a single commitDeferredSync() at the end fsyncs once for the whole batch. The then-current public batch-insert path used it; single inserts kept per-write durability. As in §4, current raw-delta code has different index work.

Under F_FULLFSYNC, throughput rises from 102 to 433 inserts/s (4.3×) as one synchronization covers more inserts; plain-fsync throughput is essentially flat. Batch 1 and Table 2 are independent runs, so their ranges carry the signal. Acknowledgments remain stable and follow the shared synchronization.

Crash scope. The stable path is exercised inside the committer's 15-frontier _exit matrix (§6); the child exits while its database object is live, and the parent reopens that database through production recovery. This tests process death and the OS-visible disk image. Separately, on AWS CentOS 9 (kernel 5.14) and ext4, dm-log-writes logged the committer's device-level writes on three 1-GiB, 512-byte-sector loop devices. Xfstests replay-log replayed every

Table 4: Symbols used by the recall-loss contract.

Symbol	Meaning
D	Durable population before the crash
U, W	Volatile tail and its size, $W = \lvert U \rvert$
$k, k_{\min}$	Query width and minimum weak-visible width
T	Frozen exact pre-crash top-k set
S	Members of U that survive recovery
M	Exposure, $\lvert T \cap U \rvert / k$
L	Membership loss, $\lvert T \cap (U - S) \rvert / k$
Δ^+	Positive recall drop after recovery
ε, δ	Loss budget and model tail probability

block-log entry 268–369, from DB_READY through the crash mark; each prefix underwent normal journal recovery and read-only verification against an external ledger with 18 intents, 18 acknowledgments, and 20 named frontiers. All 102 cuts pass: every stable acknowledgment survives, and only records in the weak window may be absent. This exercises recorded device-write prefixes beyond the OS-visible images, but on one host, filesystem, and workload rather than across storage stacks.

6 RECALL-BOUNDED DURABILITY

Standard group commit does not weaken durability: it releases a group only after the shared synchronization. Our separate, opt-in weak-ACK API may return after a complete WAL mutation becomes visible but before a fence is synchronized. Let D be the total durable population across sealed HNSWs and raw R, let $W = |U|$, and let $k_{\min}$ be the smallest query width allowed to observe U. Latest requests with $k < k_{\min}$ are served from the stable snapshot. For a query with frozen exact pre-crash top-k set T, define $M = |T \cap U|/k$.

Admission contracts. Strict admission requires

$$W + m \le \lfloor \varepsilon k_{\min} \rfloor$$

for an atomic request of m records. It is distribution-free; importantly, the cap is zero when $\varepsilon < 1/k_{\min}$, so such a configuration transparently uses stable group commit. In plain language, strict mode caps the entire volatile tail tightly enough that no query distribution can violate the loss budget. Exchangeable-mean admission instead requires $(W + m)/(D + W + m) \le \varepsilon$. If insertion order is independent of a query, $|T \cap U|$ is hypergeometric and $\mathbb{E}[M] = W/(D+W)$. This is a population-mean model, never a per-query guarantee. In plain language, it permits more weak records by assuming inserts are not unusually relevant to a query. A third policy admits only when the exact hypergeometric tail

$$\Pr[\mathrm{Hypergeom}(D + W + m, W + m, k_{\min}) > \lfloor \varepsilon k_{\min} \rfloor] \le \delta.$$

In plain language, the hypergeometric policy admits a tail only when that same random-order model puts the chance of exceeding the budget below δ. The implementation also enforces record, byte, and age limits. If a request does not fit, it fences the current tail and retries; an indivisible request that still does not fit receives a stable ACK. No policy silently forces $W \ge 1$. Therefore every servable query that includes weak records has $k \ge k_{\min}$, closing the otherwise-invalid small-k case.

Commit and query path. Followers enqueue writes to one leader, which serializes admission. For cooperating local writers using this implementation, the lifetime root lock excludes additional writable processes; within the owner, the leader is the only WAL appender. Concurrent cooperating writers therefore cannot overshoot the cap. A weak insert is released only after its complete frame and U entry are visible. A stable request, explicit fence, age/size limit, seal, or shutdown closes the generation; after successful stable synchronization, the leader atomically advances the durable frontier and reclassifies U into R. The group-delay wait is clipped to the oldest weak record's age deadline, so the worker attempts the age fence before processing queued work; device synchronization latency remains additive. In requested F_FULLFSYNC mode, failure is surfaced, telemetry records no strong-sync success, and the committer enters a failed state rather than returning Stable after plain fsync. The vectors remain in the same exact representation. The marginal weak-tail scan cost is $\Theta(Wd)$, and the total mutable exact scan is $\Theta((|R|+W)d)$, so weak admission exchanges synchronization work for query work; it is not free ANN capacity.

Exchangeability is monitored rather than assumed silently. Every latest query entry point, including segmented batch search, reports volatile hits to a one-sided CUSUM against the expected hit rate $W/(D + W)$. An alarm blocks further Exchangeable-mean admissions, fences the tail, and serves subsequent weak requests with stable ACKs. This is a drift detector, not a proof and not retroactive protection; only the configured strict policy is an adversarial weak-ACK guarantee.

Isolation invariant. Let $S \subseteq U$ be any records from the pre-crash volatile tail that are present after recovery, and let the stable candidate set and deterministic merge be identical before and after recovery. Because U never mutates an HNSW, the exact membership loss is

$$L = \frac{|T \cap (U - S)|}{k},$$
$$\Delta^+ = \max(0, \mathrm{recall}_{pre} - \mathrm{recall}_{rec}) \le L \le M.$$

We define topology amplification as $\max(0, \Delta^+ - L)$. Under isolation, loss is intentionally independent of HNSW ef: the durable topology and candidate fingerprint must remain unchanged. The evidence is therefore zero amplification plus a fingerprint check, not a fabricated ef effect.

Validation protocol. Three tests exercise different failure models. First, the child calls _exit with a live database at 15 predeclared admission, fence, and seal frontiers; all 15 read-only verification, writable reopen, and re-verification cases pass. Second, an exhaustive fixture opens all 661 byte-length prefixes of a 660-byte WAL through the production read-only recovery path; each exposes exactly the last complete fence and never its weak tail. These are byte-prefix images, not physical torn-write experiments. Third, the end-to-end benchmark uses two fixed base graphs (seeds 100 and 117), 160 clustered 12-dimensional base vectors, 25 writes from four concurrent writers, $k = 10$, 30 image executions, and 12 recovery queries for each of eight cases: 240 images, 2,880 recovery-query observations, and 14,100 raw operation rows. Each case uses each graph seed 15 times, and case order is shuffled within each repetition. Six cases clone a terminal

Table 5: Canonical Apple-silicon F_FULLFSYNC run, 30 images/case. Performance is median [min,max]; ACK p50 is weak (W) or stable (S); syncs cover the timed writer interval. The loss column is mean Δ^+ followed by image min/median/max; the last column reports maximum M and maximum L separately. The strict-hot crash and resumed suffix are outside timing. Fixed-count/time controls are omitted here but remain in all invariant totals and raw CSVs.

Policy/workload	$W_{\max}$	writes/s	ACK p50 (ms)	Syncs	mean Δ^+; image min/med/max	max M; max L
Stable group	0	326 [268,356]	S 10.85 [9.92,12.91]	7	0; 0/0/0	0; 0
Strict/random	2	251 [215,271]	W 15.74 [13.78,18.96]	12	.017; .017/.017/.017	.20; .20
Strict/hot	2	252 [200,277]	W 15.65 [14.67,19.80]	12	0; 0/0/0	.20; .00
Exch.-mean/random	9	493 [446,558]	W 6.18 [5.42,7.75]	2	.051; .017/.050/.083	.30; .30
Exch.-mean/hot	9	511 [466,587]	W 5.91 [5.04,7.89]	2	.225; .225/.225/.225	.30; .30
Exch.-mean+guard/hot	4	163 [137,172]	W 3.42 [2.90,3.83]	22	0; 0/0/0	0; 0

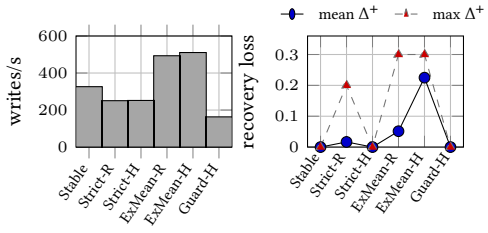

Figure 1: Median committer throughput and recovery loss from the same canonical run (min/max are in Table 5). R/H denote random/hot. Exchangeable-mean mode has a higher median but is model-conditional; the hot guard alarms, fences, and falls back.

unfenced suffix. Strict-random first fences its timed prefix, then a child acknowledges two query-targeted weak records—exactly $\lfloor .2 \cdot 10 \rfloor$—and exits before a fence. Strict-hot instead exits at the production post-sync/pre-publish failpoint after fencing two such records. The parent measures immediate recovery; for strict-hot it also performs and re-verifies a stable suffix on a copy, making that crash a nonterminal mid-workload frontier. Both controlled crash phases are outside the timed writer interval. The data are deliberately small enough to audit; they are not SIFT1M or a scale result. Hypergeometric admission is unit-tested but has no performance row in this benchmark. The six policy/workload rows displayed below plus two fixed-count/time controls make the eight cases; those controls supply the remaining 60 images and 720 query observations and remain present in the aggregate and raw CSVs.

Results. Table 5 and Fig. 1 summarize the canonical run. Every aggregate row has zero stable-record loss, unexpected weak survivor, cap overshoot, and resumed-suffix failure. All 2,880 recovery rows satisfy $\mathrm{LostIDs} \subseteq U_{pre}$ and both deterministic-merge fingerprint checks. Amplification is zero within 10^{-12} (the only nonzero machine value is 5.55×10^{-17}). In strict mode with $\varepsilon = 0.2$ and $k = 10$, every observation satisfies $\Delta^+ \le L \le M \le 0.2$. The strict-random child reaches the cap in all 30 images with $|U| = 2$, exits before fencing, and recovers no cohort members ($|S| = 0$). Thus one query per image has $M = L = \Delta^+ = 0.2$; no strict observation exceeds the cap. In all 30 strict-hot images, the child records

Table 6: Paired strict/stable throughput ratio, median [min,max], at $\varepsilon = .2$. "Faster reps" counts descriptive strict-faster timing repetitions within two fixed graphs, not independent graph samples. Every strict image binds. The final column is median timed syncs, strict/stable.

$k_{\min}$	cap	random ratio	faster reps	hot ratio	faster reps	syncs
10	2	.69 [.48,.77]	0/30	.69 [.45,.82]	0/30	31/16
20	4	1.05 [.65,1.61]	20/30	1.07 [.76,1.56]	22/30	15/16
50	10	1.40 [.96,1.59]	28/30	1.38 [1.01,1.71]	30/30	6/16
100	20	1.61 [1.03,1.84]	30/30	1.64 [1.08,2.98]	30/30	3/16

$|U| = 2$ before exiting after fence sync; recovery finds both records ($|S| = 2$), so $L = \Delta^+ = 0$ while $\max M = 0.2$. This yields 90 query observations across 30 images with $L < M$, 60 expected survivors, and 30 stable suffixes verified after writable recovery. A separate reversed-order base built with seed 1109 is rejected against the seed-100 configuration on persisted-seed metadata (with no observed answer drift); normal runs use matching persisted seeds and recovery fingerprints.

The performance tradeoff is large and noisy. Stable group commit's median is 326 writes/s; Exchangeable-mean/random's is 493, with observed ranges 268–356 and 446–558. Exchangeable-mean/hot spans 466–587. These are same-host timing ranges, not population confidence intervals. Timed synchronization counts are 7 for stable, 12 for strict, and 2 for unguarded Exchangeable-mean rows. For target $\varepsilon = .05$, random image-mean loss has mean .051, median .050, and range .017–.083; hot loss is .225 throughout. Exchangeable-mean is therefore a workload model, not a per-query promise.

This is the headline tradeoff: at $k = 10$, the adversarially safe weak mode costs throughput relative to stable commit. Exchangeable-mean raises throughput in this run only by dropping the per-query guarantee.

Binding strict sweep. We next fix $\varepsilon = .2$, use 64 writes and 64 latest queries from four writers, and vary $k = k_{\min} \in \{10, 20, 50, 100\}$. Every strict trial is adjacent to a stable control with the same base image, seeds, vectors, keys, metadata, query workload, and group delay; pair order is randomized. All 240 strict images bind, all 15,360 requests return Weak, and no cap overshoots or syncs fail. Table 6 reports paired ratios under the substantial timing noise. At $k = 10$, strict is slower in all 30 timing repetitions for both workloads because it performs 31 timed syncs versus stable's median 16. At $k = 100$, strict performs 3 syncs and reaches median 580/577 writes/s versus 362/350 for stable on random/hot; it is faster in all 30 repetitions for each workload. A binding strict regime can therefore beat stable commit once its cap amortizes fencing; at $k = 50$ it is faster in 28/30 random and 30/30 hot repetitions. These counts are descriptive within two fixed graphs (15 repetitions each), with no independent-graph inference or sign-test p-value. This is a one-host workload result, not a universal crossover.

Strict mode preserves the distribution-free bound across the full throughput ranges in Table 5. The hot guard alarms in all 30 images and has median 163 writes/s while fencing and

Table 7: Cold-open recovery time, sealed (snapshot load) vs. mutable (WAL replay), Apple M4 Pro, d=128, F_FULLFSYNC; median of 7 trials (min–max). Snapshot-creation/sealing cost is *not* included—only the cost of reading an already-sealed snapshot. WAL-replay time is dominated by graph rebuild and is fsync-mode-independent.

N	Sealed (ms)	WAL replay (ms)	Gap
1,000	38.9 (32.4–46.8)	195.5 (181.7–196.5)	5.0×
3,000	44.2 (33.8–55.0)	650.7 (647.0–683.9)	14.7×
6,000	47.0 (42.7–48.4)	1,483.6 (1,455.6–1,602.0)	31.6×

serving later writes stably; its weak p50 is 3.42 ms and stable-fallback p50 is 26.91 ms. Median query p95 is 6.37/5.59 ms for unguarded Exchangeable-mean random/hot and 24.85 ms with the guard. These are one-host ranges, not deployment claims. Exact-tail scaling is not isolated; only its $\Theta(Wd)$ marginal cost is established.

7 FAST, CRASH-CONSISTENT RECOVERY

The pre-committer stable engine supplies a complementary recovery-cost baseline. A sealed segment loads an HNSW snapshot, whereas an unsealed legacy mutable segment replays its WAL through the graph builder. This is distinct from the new committer's raw R/U recovery path. Table 7 shows that, from N = 1k to 6k, snapshot load rises from 39 to 47 ms while rebuild rises from 196 to 1,484 ms; the gap grows from 5.0× to 31.6×. Snapshot load is still linear in serialized size, with a smaller per-element constant, not constant time. A separate pre-committer plain-fsync run measured snapshot creation at 29.7/36.0/55.3 ms at the same sizes; because Table 7 uses F_FULLFSYNC, those creation numbers are context rather than a direct end-to-end comparison. The table is the seven-trial Apple result; the AWS cross-check reaches a 128× gap at 6k (§3).

Extending this pre-committer sweep to N = 20k gives 37 ms snapshot load versus 5.9 s graph rebuild, a 157× gap. These timings justify periodic sealing for the legacy mutable-HNSW design; they should not be read as a measured recovery cost for the new raw-delta committer.

8 INDEX COMPETITIVENESS

Table 8 is a sanity check that durability is not measured on a non-competitive index. On SIFT1M, recall@10 rises from .770 to .999 as ef grows; against hnswlib on the same host and parameters, QPS is within a few percent and the measured recall points are slightly higher. Because QPS is single-run and graph builds are randomized, we claim only that this build is on the same practical recall–QPS frontier, not that it is deterministically superior.

9 RELATED WORK

Dynamic and durable ANN. FreshDiskANN [14] and SPFresh [20] add streaming graph updates and describe log/snapshot recovery; LSM-VEC [23] distributes a proximity graph across LSM levels. P-HNSW [9] compares HNSW WAL replay and snapshot load on persistent memory. Our legacy measurements instead isolate synchronization cost on block storage, and our committer asks a different question: which acknowledged records may be discarded, and what top-k damage can that cause? Manu coordinates log-publishing writers and subscribing search components through WAL/binlog backbone services and delta consistency [7]; it does not state our weak-ACK recall-loss contract.

Table 8: Recall@10 vs. single-thread QPS on SIFT1M (Apple M4 Pro, d=128, n=10^6, M=16, $\mathrm{ef}_{\mathrm{construction}}$=200), ours vs. hnswlib on the identical dataset/params/host, measured against the dataset's shipped exact ground truth (ID-set recall@10). QPS is single-run, and graph builds were not repeated.

	this work		hnswlib	
ef	recall@10	QPS	recall@10	QPS
10	0.770	22,190	0.709	20,829
32	0.931	10,114	0.904	9,907
64	0.977	5,851	0.963	5,758
100	0.990	4,045	0.983	3,903
200	0.997	2,240	0.996	2,200
500	0.999	1,023	0.999	1,040

Turbopuffer is the closest operational composition [17]: successful writes are durably object-stored in a WAL, while committed but not yet indexed records remain searchable by an exhaustive tail, and a separate system monitors continuous recall [5]. That architecture separates durable commit from indexing; it does not admit an acknowledged undurable tail under a recall-loss budget. VStream studies distributed streaming vector search and the freshness/recall tradeoff [6]; it does not define a storage-failure loss contract.

Bounds and measurement. TACT [21] and Pileus [15] bound consistency or staleness; PBS bounds replica-version staleness probabilistically for partial-quorum reads [2], not membership loss after storage failure. Aether [8] and Silo [16] develop logging/group commit. ANN-Benchmarks [1], Big ANN [13], and VectorDBBench [24] omit crash recovery and synchronization modes. Relative to PBS and Turbopuffer, our contribution is an auditable ANN contract: acknowledge a not-yet-durable insert only while a strict distribution-free post-crash top-k loss cap holds on every query path, then verify that contract after production recovery. We claim neither a new ANN algorithm nor a new logging primitive.

10 THREATS TO VALIDITY AND REMAINING WORK

The legacy tax/recovery study uses seven trials on Apple and a separate AWS Linux cross-check; committer timings use one Apple M4 Pro and a small clustered synthetic workload. The physical replay uses one AWS CentOS 9/ext4 host and the same small committer path. Thirty tail/write image executions per case expose substantial timing variance, but they share one query set and only two fixed base graphs. The terminal frontier is repeated for six cases;

the cap-before-fence frontier is repeated for strict-random, and the post-sync/pre-publish frontier for strict-hot. These are not independent graph or deployment-workload samples, and several performance ranges overlap. All pre-crash recall values are 1.0, so the loss experiment validates membership isolation rather than ANN-error interaction. There is no controlled query-cost sweep in W or d. The legacy sync-floor loop also omitted syscall return-value accounting, so we use it only to compare timing under requested sync modes, not to claim observed power survival.

Five implementation threats identified during review are now closed and tested. Latest scalar, metadata, and batch queries with $k < k_{\min}$ use stable visibility; every weak-visible batch query feeds the correlation guard; the oldest-tail age deadline preempts group delay; and failed F_FULLFSYNC returns an error without a plain-fsync fallback or stable ACK. Strong-mode atomic replacement now full-syncs the temporary file, renames it, and only then full-syncs the parent directory; a reservation-rollover fault test checks descriptor order and fail-closed behavior. The age deadline determines when the worker attempts fencing, not how long the device takes to complete synchronization. Injected-I/O tests cover these paths, but not all real-device error and recovery modes.

Cooperating-writer exclusion is enforced by an exclusive writable-root OS lock acquired before recovery or storage mutation and held until close. Fork tests cover rejection while the owner is live and reacquisition after both normal close and _exit; read-only opens remain concurrent. This is an advisory-lock guarantee for cooperating processes on filesystems with local flock semantics. It does not protect against a writer that bypasses this implementation, nor establish behavior on a network filesystem with incoherent locking.

Strict admission is distribution-free but permits only $\lfloor \varepsilon k_{\min} \rfloor$ weak records; for common small $k_{\min}$ and $\varepsilon < 1/k_{\min}$, it degenerates correctly to stable commit. Exchangeable-mean mode can admit more, but hot records violate its mean model, as Table 5 demonstrates. The CUSUM limits future exposure after detection and cannot undo an already acknowledged tail. In the guard benchmark, image cloning waits for the alarm-triggered fence, so its zero-loss row does not test a crash during detection or synchronization. Total mutable search is $\Theta((|R| + W)d)$. The implementation and evaluation are insert-only, single-node, and unfiltered; updates/deletes retain stable semantics, and tenant filters, sharding, and replication remain open.

Finally, 15 _exit points test process death, 661 byte prefixes test the WAL parser, and cloned images test production reopen. The executed Linux dm-log-writes run adds 102/102 passing device-level write-prefix cuts with normal journal recovery and an external ACK oracle. Its scope is still one CentOS 9 host, ext4, loop-backed devices, and one short workload; it does not establish behavior across filesystems, controllers, or literal power removal. The older graph-rebuild recovery table is a pre-committer baseline, not a measurement of current raw-delta recovery.

11 CONCLUSION

ANN durability should be reported as a contract, not inferred from an API name. The two-host legacy study shows why: synchronization semantics change measured cost, stable group commit recovers much of that cost, and snapshot loading avoids graph-rebuilding replay. Those are established storage techniques measured on an ANN path, not new primitives.

Our central contribution is an auditable opt-in contract that acknowledges not-yet-durable inserts only under a strict distribution-free post-crash top-k loss cap enforced across every latest-query path and checked after production recovery. A lifetime root lock provides cooperating-writer exclusion on local filesystems, while an in-process leader serializes WAL appends; fenced WAL generations define recovery, and exact R/U layers keep volatile inserts out of HNSW topology. Across 15 process-crash frontiers, 661 WAL prefixes, and 2,880 recovered-query observations, stable records survive, lost IDs stay within U_{pre}, admission never overshoots, and the observed strict rows satisfy $\Delta^+ \le L \le M \le \varepsilon$. The cap-before-fence frontier loses 60/60 exposed records and reaches $M = L = \Delta^+ = .2$ in every image; the post-sync frontier supplies 60 survivors and 90 observations with $L < M$. At the default $k = 10$, strict mode is slower than stable commit. The paired sweep finds a positive but conditional systems regime: at $k = 100, \varepsilon = .2$, the 20-record cap binds and strict is faster in all 30 within-fixed-graph timing repetitions for both workloads. Exchangeable-mean mode is higher in the default run, but it fails as a per-query guarantee on hot data. The guard responds by fencing and falling back to stable ACKs at substantial cost. The shipped default remains stable. On the separate CentOS 9/ext4 host, all 102 dm-log-writes cuts also preserve the ledger contract. Generalization awaits larger workloads and additional hosts, filesystems, and storage devices.

REPRODUCIBILITY

Run the committer artifact from the repository root:

```
make test
scripts/run_recall_committer_evidence.sh \
  --repetitions 30 \
  --output .artifacts/recall-committer-evidence
scripts/run_recall_committer_evidence.sh \
  --repetitions 30 --validate-existing \
  docs/paper/data/recall_committer_canonical \
  --no-install
cd docs/paper
~/.local/bin/tectonic -X compile \
  honest-durability.tex
```

The first command runs 77 general tests. The staging script clean-builds 24 policy/API tests, the 15/15 process-crash and 661/661 byte-prefix sweeps, and both benchmarks. It validates hashes, schemas, counts, pairs, seeds, and invariants before atomic publication; the second invocation audits the tracked bundle. Neither invokes the destructive physical harness.

The executed device-level recipe was the following; it requires Linux, root, explicit opt-in, and three disposable devices (the target also builds the verifier):

```
make committer-crash-test
truncate -s 1G /tmp/vdb-{data,log,replay}.img
for p in 10:data 11:log 12:replay; do
  sudo losetup /dev/loop${p%%:*} /tmp/vdb-${p#*:}.img
done
RL=/home/cloud-user/xfstests-dev/src/log-writes/replay-log
A=$PWD/docs/paper/data/powerloss_committer
sudo env VDB_POWERLOSS_ALLOW_DESTRUCTIVE=YES \
```

```
DATA_DEV=/dev/loop10 LOG_DEV=/dev/loop11 \
REPLAY_DEV=/dev/loop12 REPLAY_LOG=$RL \
ARTIFACT_DIR=$A \
scripts/powerloss_committer.sh
```

The x86-64 CentOS 9 host ran kernel 5.14.0-710.el9.x86_64, ext4 (mke2fs 1.46.5), and device-mapper library/driver 1.02.207-RHEL9/4.50.0. The 102/102 PASS bundle (entries 268–369) is under docs/paper/data/powerloss_committer/; ledger SHA-256 is e9ac3e910b1728b21c825273bc09e79a 0013f430d4aa1bb5ddb247c78c56b6b3.

The timing run used an Apple M4 Pro, Darwin 25.5.0, APFS, Apple clang 21.0.0, C++20 -O2 -mcpu=native, and F_FULLFSYNC. Defaults are 160 base records, 25 writes, 32 queries, $d = 12$, $k = 10$, four writers, 30 repetitions, ef = 32, and graph seeds 100/117. Evidence from code commit 9d93ba5fceab945985c28e9f18e337bf2c7c1555 (run 20260720T055159Z-9d93ba5fceab) is tracked under docs/paper/data/recall_committer_canonical. It contains 240/2,880/14,100 aggregate/query/operation rows and 480/30,720 paired-sweep images/operations. The compact paired table is data/recall_committer_canonical/recall_committer_throughput_sweep_summary.csv. Archived pre-committer tax/group/recovery CSVs are authenticated by their SHA-256 manifest. Current make bench-durability exercises the raw-delta engine and is not their exact reproducer. Their provenance record pins commit 3f8d2cce53d4c4966f996969cf1a8cfaa282cf85 while disclosing missing raw trials and producing-tree metadata. Table 8 uses make bench-ann and make bench-hnswlib; the data README records SIFT1M hashes and pins hnswlib v0.9.0 commit d9b3608c83d83b46c96e25088cb1d729b29dcfe9.

REFERENCES

[1] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. *Information Systems* 87 (2020), 101374. https://doi.org/10.1016/j.is.2019.02.006

[2] Peter Bailis, Shivaram Venkataraman, Michael J. Franklin, Joseph M. Hellerstein, and Ion Stoica. 2012. Probabilistically Bounded Staleness for Practical Partial Quorums. *Proceedings of the VLDB Endowment* 5, 8 (2012), 776–787. https://doi.org/10.14778/2212351.2212359

[3] Jonathan Corbet. 2018. PostgreSQL's fsync() Surprise. LWN.net. https://lwn.net/Articles/752063/

[4] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss Library. arXiv:2401.08281

[5] Maxime Gallant. 2024. Continuous Recall Measurement. Turbopuffer Engineering Blog. https://turbopuffer.com/blog/continuous-recall

[6] Shuang Gong, Haoyu Sun, Zhen Fang, Li Liu, Lei Chen, and Yunjun Gao. 2025. VStream: A Distributed Streaming Vector Search System. *Proceedings of the VLDB Endowment* 18, 6 (2025), 1593–1606. https://doi.org/10.14778/3725688.3725692

[7] Rui Guo, Xiaoliang Luan, Long Xiang, Xiao Yan, Xiaomeng Yi, Jun Luo, Qianya Cheng, Weizhi Xu, Jiarong Luo, Frank Liu, Zhenshan Cao, Yanliang Qiao, Ting Wang, Bo Tang, and Charles Xie. 2022. Manu: A Cloud Native Vector Database Management System. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3548–3561. https://doi.org/10.14778/3554821.3554843

[8] Ryan Johnson, Ippokratis Pandis, Radu Stoica, Manos Athanassoulis, and Anastasia Ailamaki. 2010. Aether: A Scalable Approach to Logging. *Proceedings of the VLDB Endowment* 3, 1 (2010), 681–692. https://doi.org/10.14778/1920841.1920928

[9] Hyeontaek Lee, Taewoo Park, Youngjae Na, and Wook-Hee Kim. 2025. P-HNSW: Crash-Consistent HNSW for Vector Databases on Persistent Memory. *Applied Sciences* 15, 19 (2025), 10554. https://doi.org/10.3390/app151910554

[10] Yu A. Malkov and D. A. Yashunin. 2016. hnswlib: A Header-Only C++/Python Library for Fast Approximate Nearest Neighbor Search. GitHub repository, nmslib/hnswlib. https://github.com/nmslib/hnswlib Apache-2.0; maintained through the present.

[11] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. https://doi.org/10.1109/TPAMI.2018.2889473

[12] Thanumalayan Sankaranarayana Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswany, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2014. All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Broomfield, CO, USA, 433–448.

[13] Harsha Vardhan Simhadri, Martin Aumüller, Amir Ingber, Matthijs Douze, et al. 2024. Results of the Big ANN: NeurIPS'23 Competition. arXiv:2409.17424

[14] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. arXiv:2105.09613

[15] Douglas B. Terry, Vijayan Prabhakaran, Ramakrishna Kotla, Mahesh Balakrishnan, Marcos K. Aguilera, and Hussam Abu-Libdeh. 2013. Consistency-Based Service Level Agreements for Cloud Storage. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*. Association for Computing Machinery, New York, NY, USA, 309–324. https://doi.org/10.1145/2517349.2522731

[16] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. 2013. Speedy Transactions in Multicore In-Memory Databases. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*. Association for Computing Machinery, New York, NY, USA, 18–32. https://doi.org/10.1145/2517349.2522713

[17] Turbopuffer. 2026. Architecture: Durable Writes and Unindexed-Data Search. Product documentation. https://turbopuffer.com/docs/architecture Accessed July 2026.

[18] Naman Upreti, Harsha Vardhan Simhadri, H. S. Sundar, K. Sundaram, et al. 2025. Cost-Effective, Low-Latency Vector Search with Azure Cosmos DB. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5166–5183.

[19] Jianguo Wang, Xiaomeng Yi, Rui Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yifan Zou, Jiachang Long, Yihua Cai, Zhen Li, Zhifeng Zhang, Yihong Mo, Jun Gu, Rui Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 2614–2627. https://doi.org/10.1145/3448016.3457550

[20] Yuming Xu, Hengyu Liang, Jin Li, Shicheng Xu, Qi Chen, Qian Zhang, Cheng Li, Ziyi Yang, Fan Yang, Yu Yang, Peng Cheng, and Mao Yang. 2023. SPFresh: Incremental In-Place Update for Billion-Scale Vector Search. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*. Association for Computing Machinery, New York, NY, USA, 545–561. https://doi.org/10.1145/3600006.3613166

[21] Haifeng Yu and Amin Vahdat. 2002. Design and Evaluation of a Conit-Based Continuous Consistency Model for Replicated Services. *ACM Transactions on Computer Systems* 20, 3 (2002), 239–282. https://doi.org/10.1145/566340.566342

[22] Mai Zheng, Joseph Tucek, Dachuan Huang, Feng Qin, Mark Lillibridge, Elizabeth S. Yang, Bill W. Zhao, and Shashank Singh. 2014. Torturing Databases for Fun and Profit. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Broomfield, CO, USA, 449–464.

[23] Shijie Zhong, Dingheng Mo, and Siqiang Luo. 2025. LSM-VEC: A Large-Scale Disk-Based System for Dynamic Vector Search. arXiv:2505.17152

[24] Zilliz. 2023. VectorDBBench: A Benchmark Tool for Vector Databases. Open-source software. https://github.com/zilliztech/VectorDBBench